

Advanced Classification Models & Hyperparameter Tuning

Dr. Marcelo Feighelstein

Learning Objectives

1

Master Advanced Classifiers

Random Forest, Gradient Boosting, and SVM — three powerful algorithms for classification tasks

2

Cross-Validation

Apply k-fold cross-validation for robust, unbiased model evaluation

3

Hyperparameter Optimization

Use GridSearchCV and RandomizedSearchCV for automated hyperparameter search

4

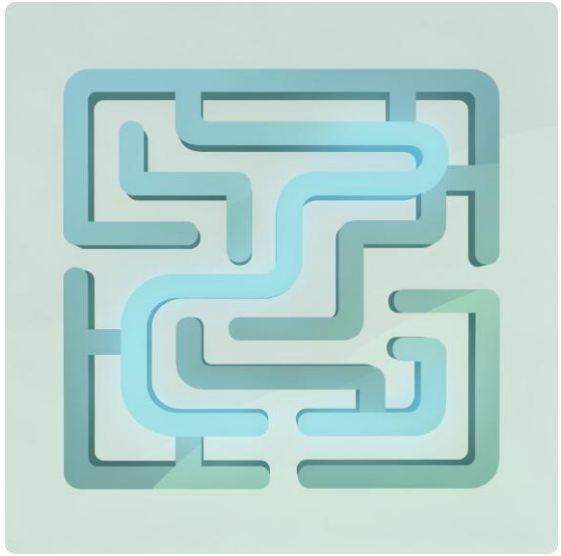
AI-Assisted Coding

Prompt Claude and ChatGPT effectively for ML code generation and iteration

5

True Generalization

Avoid overfitting and achieve models that perform reliably on unseen data



The Challenge of Model Selection

Problem 1

Which algorithm performs best on your specific data?

Problem 2

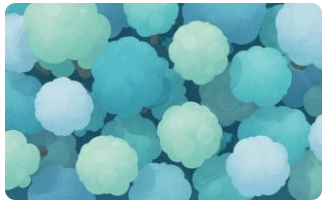
How to find optimal hyperparameters without endless manual testing?

Problem 3

How to ensure your model generalizes to unseen data?

Solution: Combine advanced algorithms with intelligent search strategies. Better algorithms + better validation = better predictions.

Three Powerful Classification Algorithms



Random Forest

Ensemble of decision trees with parallel training. Handles feature interactions naturally and provides built-in feature importance scores.



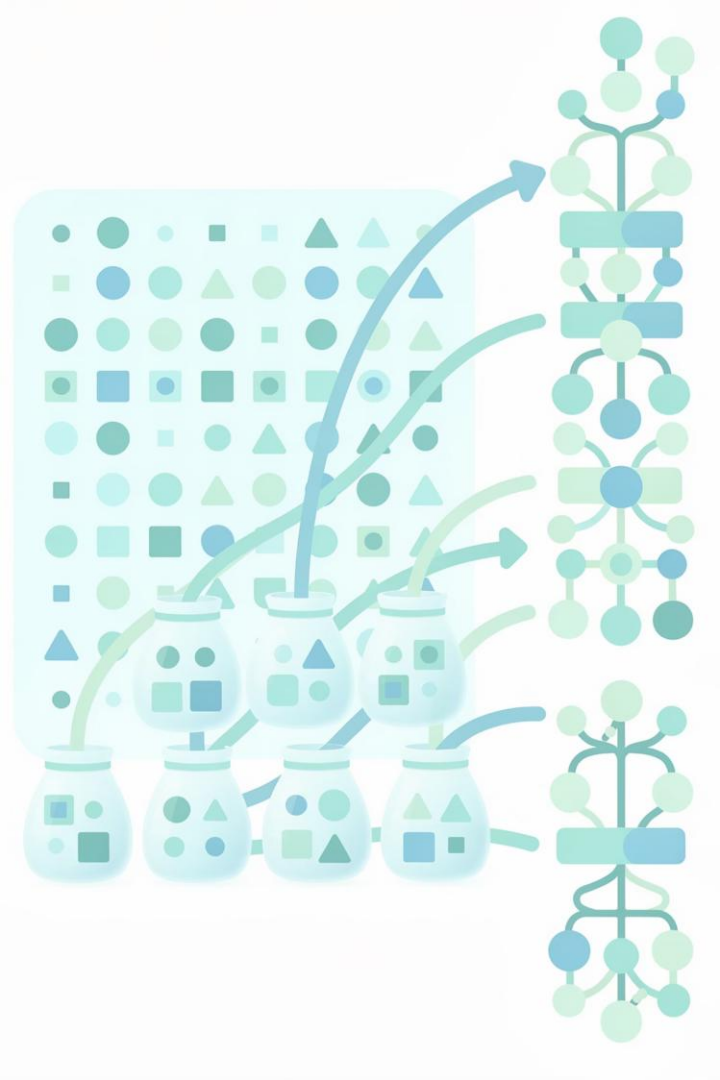
Gradient Boosting

Sequential tree building — each tree corrects previous errors. Often achieves the highest accuracy of the three approaches.



Support Vector Machine

Finds the optimal separating hyperplane. Effective in high-dimensional spaces; the kernel trick enables non-linear classification.



Random Forest: Bootstrapping (Bagging)

Each tree is trained on a unique random subset of the original training data, drawn with replacement. This introduces diversity among the trees, helping to reduce variance and prevent overfitting to the training set.

Individual Trees

Each trained on one sample

Original Dataset

Full training data

Bootstrapped Sampling Flow

Random Samples (2)

Independent bootstrap draw

Random Samples

Drawn with replacement

Random Forest: Random Feature Selection

Each tree is trained on a unique random subset of the original training data, drawn with replacement. This introduces diversity among the trees, helping to reduce variance and prevent overfitting to the training set.

01

All Features Available

02

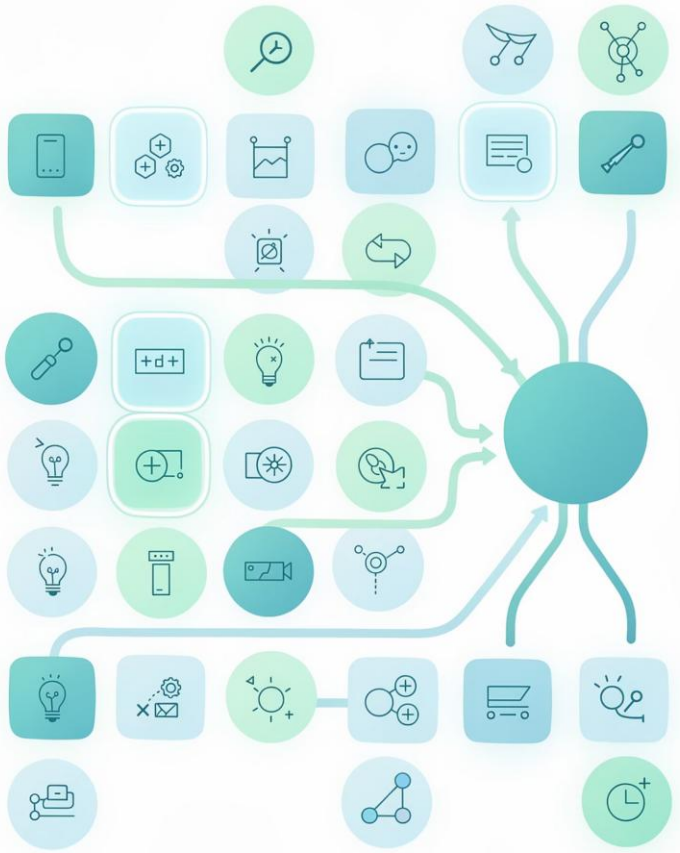
Random Subset Selected

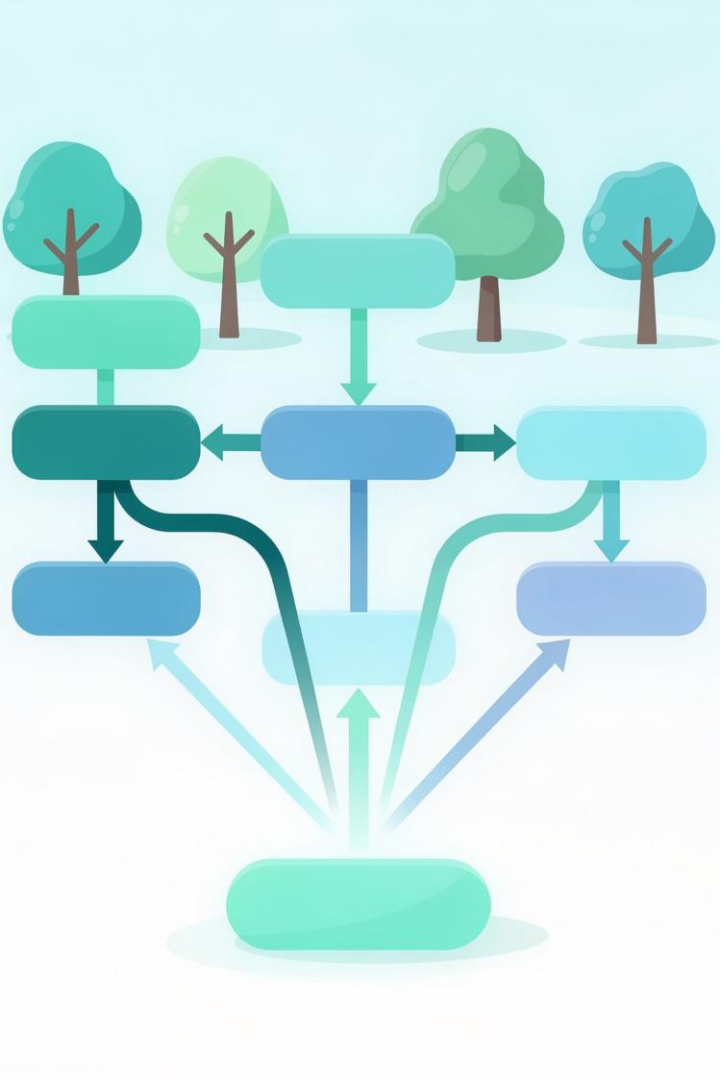
03

Best Split Found

04

Node Created





Random Forest: Ensemble Voting

For classification tasks, the final prediction is determined by a majority vote from all individual trees in the forest. For regression, the predictions are averaged. This aggregation leads to more stable and accurate outcomes than any single tree could achieve.

1. **Tree 1: Class A**
2. **Tree 2: Class A**
3. **Tree 3: Class B**
4. **Majority Vote**
5. **Final Prediction: Class A**

Random Forest: Key Hyperparameters

Strengths: handles non-linear patterns, provides feature importance.



n_estimators

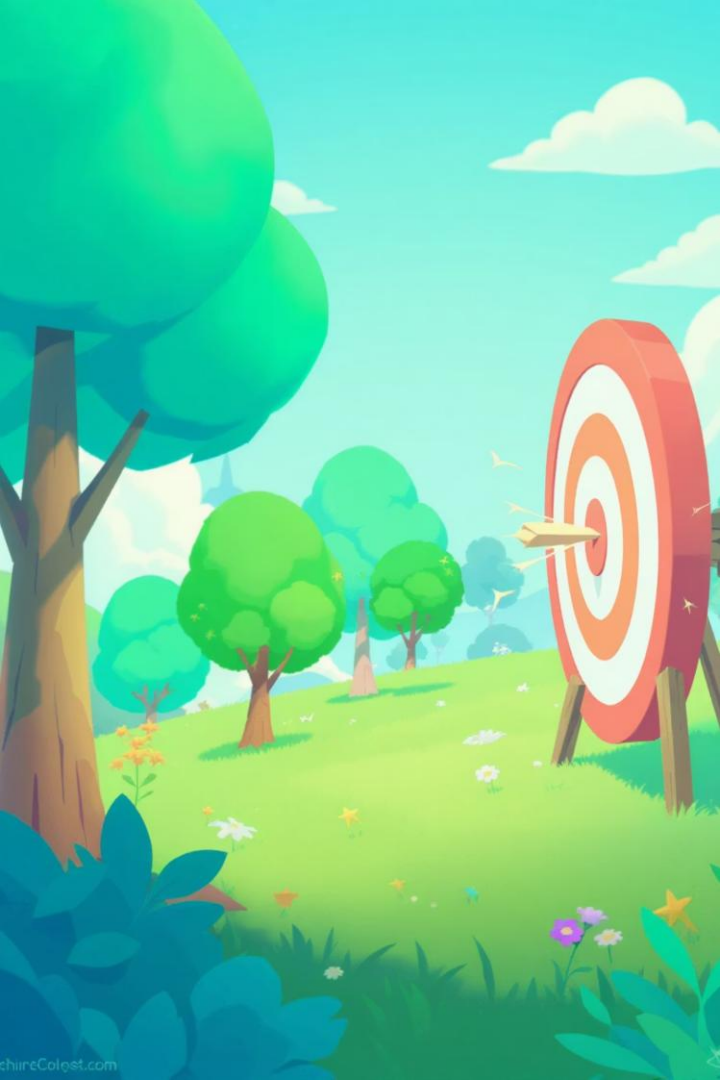
Number of trees — higher is better but slower to train

max_depth

Maximum tree depth — primary lever for controlling overfitting

max_features

Features considered per split — `"sqrt"` (√n), `"log2"`, or a specific fraction



SEQUENTIAL LEARNING

Gradient Boosting: Sequential Learning

Gradient Boosting builds an ensemble of weak learners sequentially. Unlike Random Forest, which trains trees in parallel, each new tree is trained specifically to correct the residual errors of the previous ensemble. This iterative refinement process progressively reduces prediction error.

01

Train initial tree on the data

02

Calculate residual errors

Identify what the model got wrong.

03

Train next tree on the residuals

04

Add the new tree to the ensemble

05

Repeat until `n_estimators` is reached

Residuals are the difference between the actual value and the current ensemble's prediction: `residual = actual - predicted`.
The next tree is not trained on the original labels. It is trained on these residual errors, so it learns to predict *how wrong* the current model is.
Example: if the model predicts `0.6` for a patient who actually has heart failure (`1.0`), the residual is `0.4` — the next tree learns to predict that `0.4` gap.
This is why boosting is so powerful: each tree specializes in fixing the specific mistakes of the previous ensemble.

Each tree is a weak learner — the power comes from combining hundreds of them.



Gradient Boosting: Combined Decision & Learning Rate

The final prediction of a Gradient Boosting model is the weighted sum of all individual tree predictions. The learning rate controls how much each tree contributes to this combined decision — a critical balance between speed and accuracy.

$$FinalPrediction = Tree_1 \times lr + Tree_2 \times lr + Tree_3 \times lr + \ldots + Tree_n \times lr$$

Low Learning Rate (e.g. 0.01)

Each tree contributes very little. Needs many trees (800+). More stable, better generalization, slower to train.

High Learning Rate (e.g. 0.1–0.3)

Each tree contributes more. Fewer trees needed. Faster training but higher risk of overfitting.

Best practice: use a low learning rate (0.01–0.05) with a high n_estimators (500–1000) for optimal results.

Gradient Boosting: Key Hyperparameters

1

n_estimators

Boosting rounds — 800+ common for best results

2

learning_rate

Each tree's contribution (0.001–0.1) — lower rate needs more estimators but is more stable

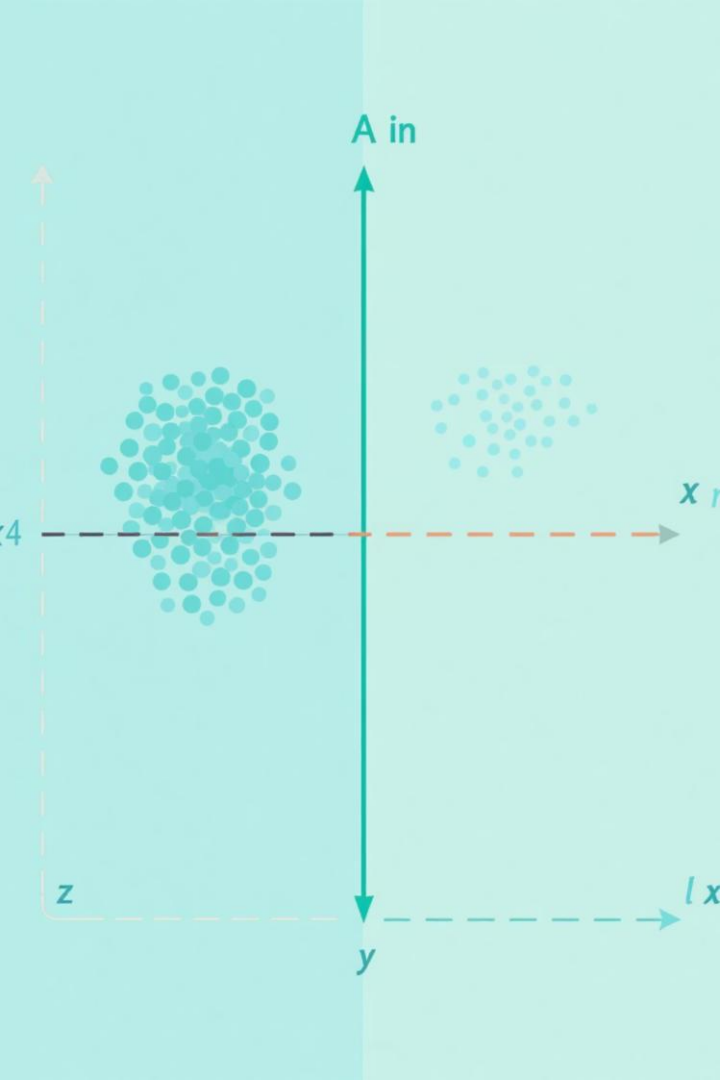
3

max_depth

Depth per tree, typically 3–8 for best generalization

Algorithm flow: build tree → calculate residuals → build next tree on residuals → combine all trees.





CORE CONCEPT

SVM: The Separating Hyperplane

Support Vector Machines (SVMs) aim to find the optimal hyperplane that best separates different classes in the feature space. The goal is not just to separate the data, but to maximize the **"margin"** — the distance between the hyperplane and the closest data points of each class.

Hyperplane

A decision boundary that separates data points of different classes. In 2D, it's a line; in higher dimensions, it's a plane.

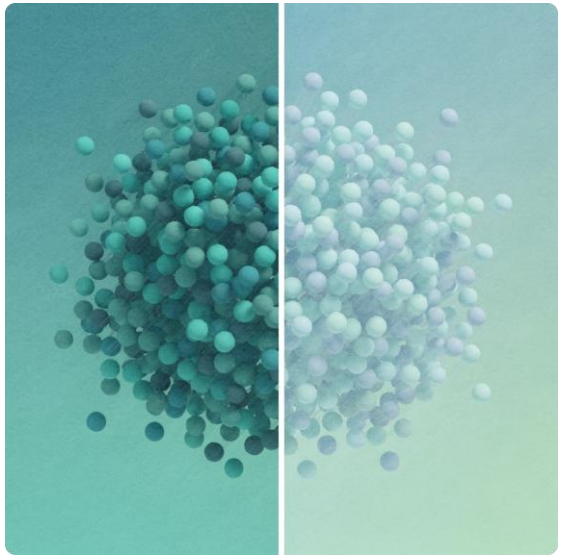
Margin

The region between the hyperplane and the closest data points from each class. SVM seeks to maximize this distance for better generalization.

Support Vectors

The data points closest to the hyperplane. These critical points define the margin and the hyperplane itself.

By focusing only on these support vectors, SVM can effectively classify even complex datasets.



SVM: Key Hyperparameters

C (Regularization)

Low C = wider margin, allows misclassifications. High C = narrow margin, overfitting risk.

kernel

`linear`, `rbf` (non-linear), `polynomial`, or `sigmoid`

Heart Failure Dataset: Model Comparison

Benchmarking four classifiers on the same dataset reveals how algorithm choice and hyperparameter tuning drive performance.

Algorithm	Accuracy	Recall	Key Hyperparameters
Decision Tree	84.78%	82.35%	max_depth=3
Random Forest	90.22%	93.46%	max_depth=6, n_est=200
Gradient Boosting	**91.30%**	**92.81%**	max_depth=6, n_est=800, lr=0.01
SVM	88.41%	92.16%	C=8, kernel=sigmoid

Key Takeaway: Gradient Boosting achieved the best accuracy — but hyperparameters make all the difference. The gap between Decision Tree (84.78%) and Gradient Boosting (91.30%) is entirely due to algorithm and tuning choices.

The Overfitting Problem

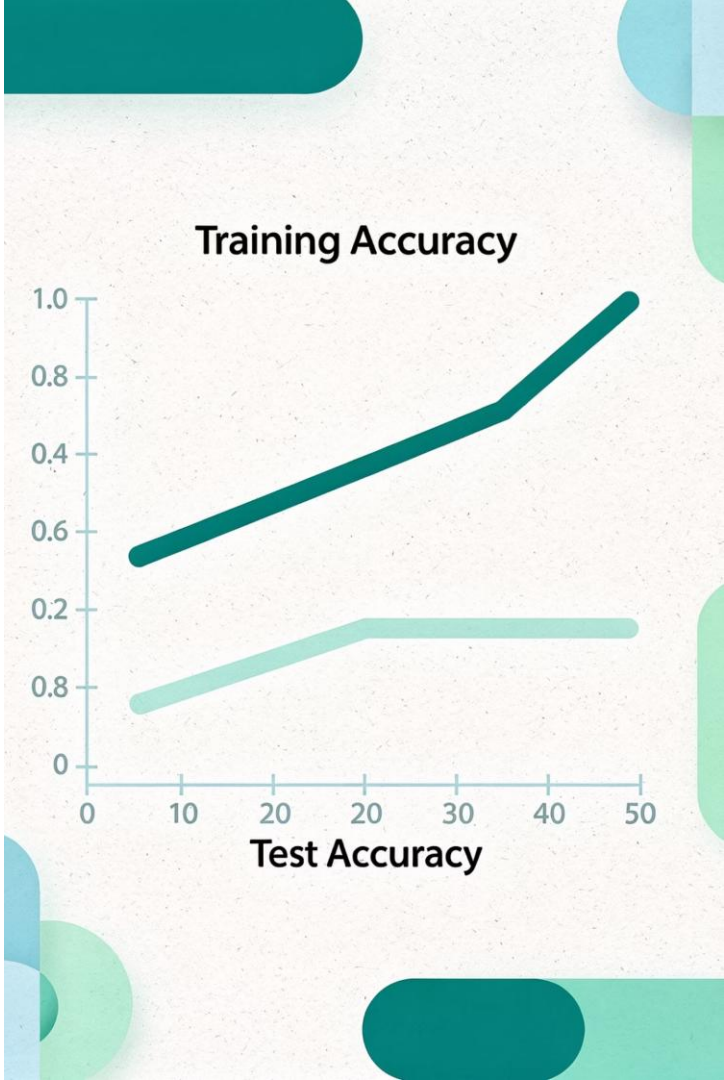
The Warning Sign

Training accuracy: **98%**
Test accuracy: **72%**
Gap: **26 percentage points**
Your model has memorized the training data — not learned generalizable rules.

Why It Happens

- Model memorizes training data instead of learning generalizable rules
- Consequences: poor real-world performance and false confidence in model quality
- Especially common with deep trees and no regularization

Solution: Cross-validation to assess true generalization before deployment.



K-Fold Cross-Validation



Divide data into **k equal folds**; use k−1 for training, 1 for testing — repeat k times. Final score = average across all k iterations → more stable, less biased than a single split. With 100 samples and 5-fold CV: 5 rounds of 80 train / 20 test. Key advantage: uses **ALL data** for both training and testing, reducing overfitting risk.

GridSearchCV: Exhaustive Hyperparameter Search

Tests Every Combination

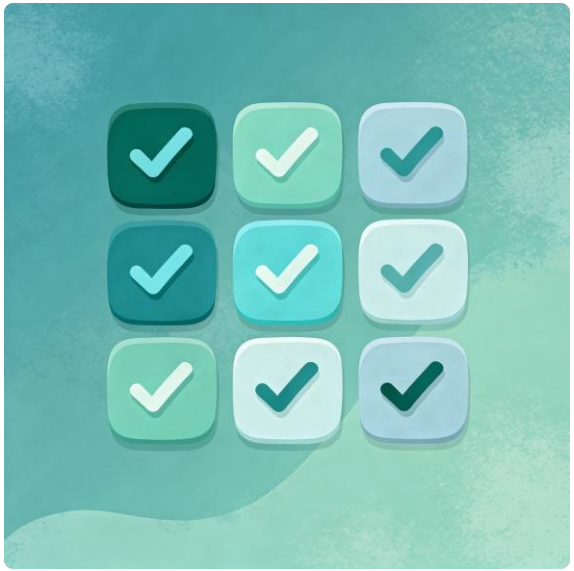
Evaluates every combination in the parameter grid with cross-validation — guaranteed to find the best combo within the grid

Scale of Search

Example: 3 × 4 × 2 = **24 combinations** × 5-fold CV = **120 model trainings**

Automatic Results

Returns `best_params_` and `best_score_` automatically — no manual tracking needed



⚠ **Con:** Computationally expensive with large grids. Use RandomizedSearchCV first to narrow the search space.

GridSearchCV: Python Implementation

Define the Parameter Grid

```
param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 7, 9],
    'max_features': ['sqrt', 'log2']
}
```

Create & Fit

```
grid_search = GridSearchCV(
    model, param_grid,
    cv=KFold(n_splits=5),
    scoring='accuracy',
    n_jobs=-1
)
grid_search.fit(X_train, y_train)
```

Access Results & Predict

```
print(grid_search.best_params_)
print(grid_search.best_score_)
y_pred = grid_search.best_estimator_.predict(X_test)
```

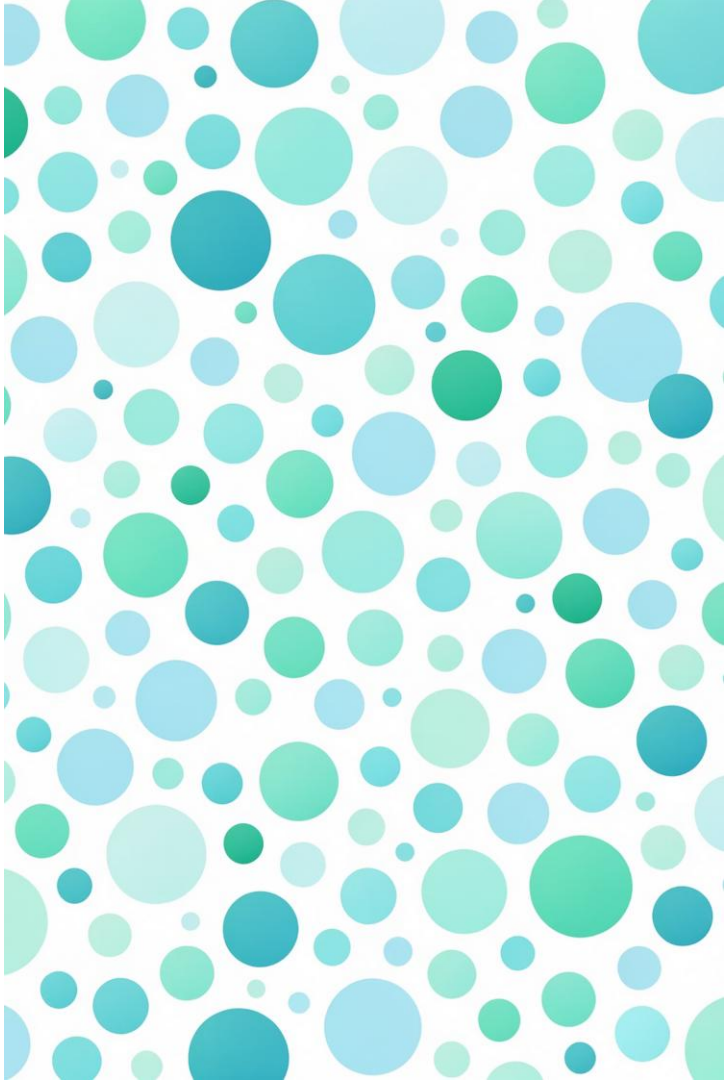
RandomizedSearchCV: Efficient Exploration

GridSearchCV

Tests **ALL** combinations
5 params × 10 values = **100,000 combos**
⏱️ Hours of compute time

RandomizedSearchCV

Tests a **random sample**
100 combos from 100,000 possible = **minutes**
Uses **distributions** (`randint`, `uniform`) instead of fixed lists for continuous ranges
Best for: large grids, continuous parameters, limited compute budget



RandomizedSearchCV: Python Implementation

Define Distributions

```
from scipy.stats import
randint, uniform

param_dist = {
    'n_estimators':
randint(50, 500),
    'learning_rate':
uniform(0.001, 0.1),
    'max_depth': randint(3,
8)
}
```

Create & Fit

```
random_search =
RandomizedSearchCV(
    model, param_dist,
    n_iter=50,
    cv=KFold(5),
    random_state=42,
    n_jobs=-1
)
random_search.fit(X_train,
y_train)
```

Access Results

```
print(random_search.best_para
ms_)
y_pred =
random_search.best_estimator_
.predict(X_test)
```

`n_iter=50` controls how many random combos to test. Access results identically to GridSearchCV.

EVALUATION

Evaluating Final Model Performance

Split the data into 80% train / 20% test before any fitting. The test set is never seen during training and provides an honest estimate of performance. Always evaluate using the `best_estimator_` on `X_test` only.

⚠ Set the test set aside before preprocessing, feature selection, or tuning to avoid data leakage.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report

# Get the best model from the search
best_model = random_search.best_estimator_

# Make predictions on the test set
y_pred = best_model.predict(X_test)

# Calculate individual metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")

# Comprehensive report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```



Addressing Persistent Overfitting

If overfitting persists after tuning, check your GridSearchCV or RandomizedSearchCV setup first.

Parameter Ranges Too Narrow

best_param_ at the edge of your grid means the optimum is outside it. Use wide ranges first: [3, 5, 7, 10, 15, None].

CV Data Leakage

Preprocessing before GridSearchCV contaminates test folds. Wrap it in a Pipeline.

Scoring Metric Mismatch

Accuracy hides failures on imbalanced data. Use scoring='f1' or scoring='recall' instead.

n_splits Too Low

cv=2 or cv=3 gives unstable estimates. Use cv=5 (standard) or cv=10 (reliable).

⚠ Set the test set aside before preprocessing or tuning to avoid leakage.

Prompting AI Tools for ML Code



Be Specific

Name the model, library, dataset size, and target metric

Add Context

"1,000 samples, 12 features, binary target, class imbalance 70:30"

Set Constraints

"Must train in <5 minutes, need >90% recall"

Request Format

"Complete notebook with comments, visualizations, and interpretation guide"

Iterate

Follow up — "Why this parameter range?" / "How do I interpret these results?"

AI Prompt Engineering: Good vs. Bad

❌ Vague Prompts

- "How do I use machine learning?"
- No data context provided
- "Give me code"

✅ Effective Prompts

- "Tune Random Forest with GridSearchCV on a 1,000-sample dataset"
- "100k samples, 50 features, imbalanced 70:30 classes"
- "Complete notebook with comments, charts, and interpretation tips"

Progression strategy: Start broad → add GridSearchCV → visualize results → explain `learning_rate` impact → iterate toward production-ready code.

End-to-End ML Pipeline



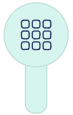
Prepare Data

Handle missing values, encode categoricals, 80/20 train/test split



Choose Algorithm

Random Forest for fast baseline → Gradient Boosting for best accuracy



GridSearchCV

3–5 values per param, 5-fold CV, identify promising ranges



RandomizedSearchCV

Expand ranges, test more combos efficiently with distributions



Final GridSearchCV

Narrow down, fine-tune with smaller increments around best params



Evaluate & Deploy

Test on holdout set, visualize importance, save model with joblib

Common Mistakes to Avoid

❌ Evaluating on Training Data

Always use a held-out `X_test` for final metrics — never report training accuracy as model performance

❌ Ranges Too Narrow

Use `[3,5,7,10,15]` not `[5,6,7]` — narrow ranges miss the true optimum entirely

❌ No random_state

Always set `random_state=42` for reproducibility — results must be consistent across runs

❌ Wrong Metric

On imbalanced data, optimize F1 or recall — not raw accuracy, which can be misleading



Key Takeaways & Next Steps

Algorithm Choice

Random Forest for speed, Gradient Boosting for accuracy, SVM for small feature sets

Always Cross-Validate

k=5 or k=10 folds for statistically reliable, unbiased estimates

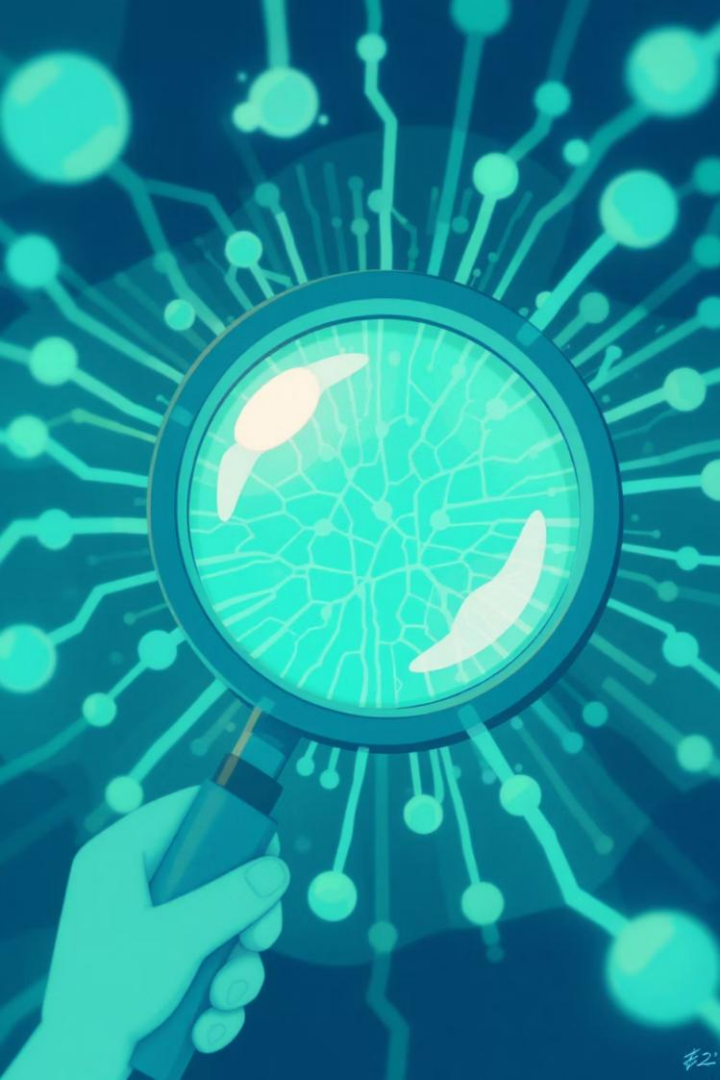
Search Strategy

RandomizedSearchCV to explore → GridSearchCV to refine

Leverage AI Tools

Structured prompts + iteration = faster, better code





INTERPRETABILITY

Understanding Model Decisions: Feature Importance

After identifying optimal models through tuning, the next critical step is to understand *why* they make certain predictions. Feature importance quantifies the contribution of each input variable to the model's overall output.

Tree-based Models (RF, GB)

Offer built-in mechanisms (e.g., Gini impurity, permutation importance) to rank features by their impact on predictions.

Provides clear, actionable insights into which variables drive the model's decision-making process.

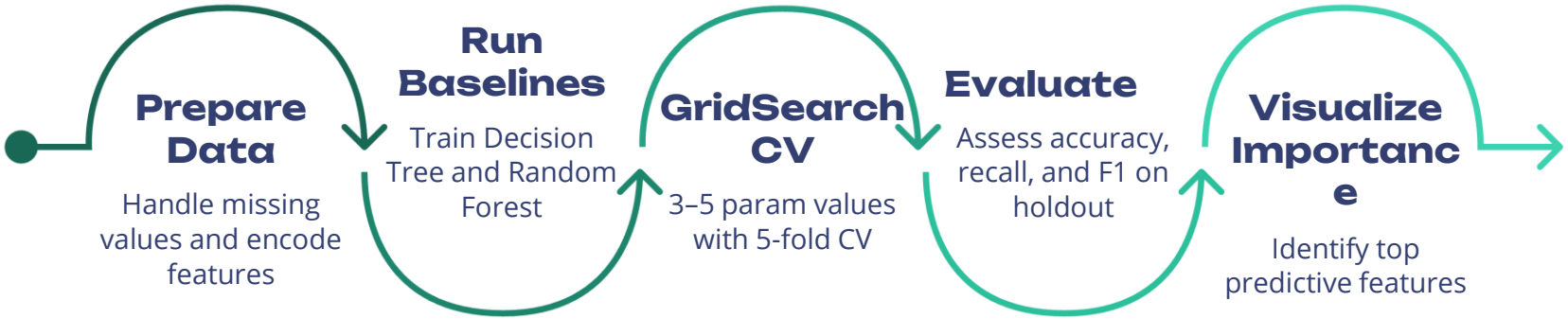
SVM & Others

Less direct interpretability. Linear SVM coefficients provide some insight, but non-linear kernels make direct feature contribution opaque.

Requires advanced post-hoc techniques (like SHAP or LIME) for interpretability, which can be computationally intensive.

Prioritize model interpretability when transparency and explainability are crucial for deployment and stakeholder trust.

Your Action Plan



Follow this pipeline on your next project: prepare your dataset → run baseline models → implement GridSearchCV → evaluate on holdout → visualize feature importance → deploy. Each step builds on the last, and the result is a production-ready, explainable ML model.

Thank You!

Go explore data — get your hands dirty, break things, and learn by doing.

Use chatbots to help you code and explore — but guide them intelligently and always judge their results critically.

📧 Email — MarcelFei@telhai.ac.il

🕐 Office Hours — Per Request

📍 Location — Building 6, Room 309, Eastern Campus